CH5: Common Mistakes in Declarations and Definitions

1. Using the wrong pointer type for a 2D array

In C, a two-dimensional array does not decay to int*.
It decays to a pointer to an array.
Using the wrong type causes incorrect row jumps and undefined behavior.

Wrong declaration:

```
int a[2][3];
int *p = a;          //  wrong: type mismatch, p is int*, a is int (*)[3]
```

Correct declaration:

```
int a[2][3];
int (*p)[3] = a;     //  correct: pointer to array of 3 ints
```

- **Always match the pointer's inner dimension with the array's column count.**
- **Otherwise, p+1 jumps to the wrong location.**

2. Incorrect assumptions about decay of 2D array names

Programmers often believe arr becomes int*. This is only true for 1D arrays.

Actual decay:

```
arr → pointer to arr[0]
arr → type: int (*)[COLUMNS]
```

Example:

```
int arr[10][4];
int *p1 = arr;         //  wrong
int (*p2)[4] = arr;    //  correct
```

3. Mixing up row address and element address

arr[i] and &arr[i][0] hold the same value, but they have different types.

Expression	Type	Meaning
arr[i]	**int[4] → decays to int***	**Address of row *i***
&arr[i][0]	**int***	**Address of first element**
&arr[i]	**int (*)[4]**	**Address of the entire row**

Common mistake:

```
int (*p)[4] = &arr[1][0]          //wrong type
```

Correct:

```
int (*p)[4] = &arr[1]             //address of whole row
```

4. Dereferencing uninitialized row pointers

A row pointer must point to a valid row before use.

Wrong:

```
int (*p)[3];
p[1][2] = 5;            //  p is uninitialized → undefined behavior
```

Correct:

```
int a[2][3];
int (*p)[3] = a;
p[1][2] = 5;            //  valid
```

- Always assign row pointers to actual array rows.

5. Using incorrect dimensions in initialization

Wrong:

```
int arr[2][3] = {
    {1,2},        //  missing element, but allowed (fills 0)
    4,5,6,7       //  invalid: excess elements
};
```

Correct:

```
int arr[2][3] = {
    {1,2,3},
    {4,5,6}
};
```

- A row cannot contain more elements than its defined width.

6. Incorrect pointer arithmetic on 2D arrays

Using a base-type pointer (int*) on a 2D array requires careful indexing.
Otherwise, memory layout gets misinterpreted.

Wrong:

```
int arr[2][3];
int *p = arr;       //  p sees memory as linear with no row structure
p += 1;             // moves only 1 int
```

Correct:

```
int arr[2][3];
int (*p)[3] = arr;  //  moves by whole row
p += 1;             // jumps to next row
```

7. Treating a string row as a full string without space for '\0'

Wrong:

```
char msg[3][5] = { "Hello", "Test" };  //  "Hello" needs 6 bytes
```

Correct:

```
char msg[3][6] = { "Hello", "Test" };  // ✓ room for '\0'
```

- Each row must have space for the null terminator.

8. Forgetting that string fields are rectangular

All rows must have equal length, even if strings differ.

Example:

```
char sys_msg[6][31];   // every message = max 31 chars
```

Common mistake: expecting variable-length rows.

Mistake	Cause	Fix
Using int* for 2D array	Wrong decay assumption	Use int (*p)[cols]
Wrong inner dimension	Pointer jumps wrong	Must equal column count
Using &arr[i][0] instead of &arr[i]	Type mismatch	Use &arr[i]
p[i][j] before p initialized	Uninitialized pointer	Assign p = arr
Overfilling initialization row	Too many elements	Match width
Linear pointer on 2D array	Missing row structure	Use pointer-to-array
Missing '\0' space	Row too small	Increase row length

Note: Determining Sizes of Multidimensional Arrays Using sizeof

The sizeof operator provides a portable and reliable way to obtain the size of elements, rows, and the entire array. The following expressions demonstrate how to calculate these values for a two-dimensional array:

```
#define ROW 10
#define COL 10

WORD arr[ROW][COL];

size_t element_size = sizeof(arr[0][0]);            // size of a single element
size_t row_size     = sizeof(arr[0]);               // size of one full row
size_t total_size   = sizeof(arr);                  // total size of the entire
matrix

size_t total_elems  = sizeof(arr) / sizeof(arr[0][0]);    // total number of
elements
size_t row_elems    = sizeof(arr[0]) / sizeof(arr[0][0]);  // number of elements
per row
```

- These calculations automatically adapt if the data type or array dimensions change, ensuring full portability without hard-coded values.

Note: Understanding Row Count vs. Index Values in 2D Arrays

In a two-dimensional array, the numbers used in indexing (e.g., arr[i][j]) represent **index positions**, not the size of the array. The table below illustrates the relationship:

Declaration	Number of Rows	Row Indices
intarr[3][5]	3	0, 1, 2
intarr[4][6]	4	0, 1, 2, 3

When writing:

```
arr[3][4] = 10;
```

- The numbers 3 and 4 are index values, not the dimensions of the array.

Example:

```
int arr[10][10];
arr[3][4] = 10;
```

Here:

- 3 → refers to the 4th row (rows: 0, 1, 2, 3)
- 4 → refers to the 5th column (columns: 0, 1, 2, 3, 4)

These numbers indicate the specific element being accessed. They do not describe or change the array's size; they only select which position inside the already defined array is being modified.